PS4 Due 3/14

1) a) substituting with the weight and errors given, we get $\underset{(h_t(x), \beta_t)}{\operatorname{argmin}} (e^{\beta_t} - e^{-\beta_t}) \epsilon_t + e^{-\beta_t}$

$$\frac{\partial L}{\partial \beta_t} = \epsilon_t (e^{\beta_t} + e^{-\beta_t}) - e^{-\beta_t} = 0$$

$$\epsilon_t (e^{\beta_t} + e^{-\beta_t}) = e^{-\beta_t}$$
$$e^{\beta_t} (e^{\beta_t} + e^{-\beta_t}) = \frac{1}{\epsilon_t}$$
$$e^{2\beta_t} + 1 = \frac{1}{\epsilon_t}$$
$$e^{2\beta_t} = \frac{1}{\epsilon_t} - 1$$
$$\ln(e^{2\beta_t}) = \ln\left(\frac{1}{\epsilon_t} - 1\right)$$
$$2\beta_t = \ln\left(\frac{1}{\epsilon_t} - 1\right)$$
$$\boxed{\beta_t = \frac{1}{2} \ln\left(\frac{1}{\epsilon_t} - 1\right)} \qquad \beta_t = \frac{1}{2} \ln\left(\frac{1}{\sum_n w_t(n) \mathbb{I}[y_n \neq h_t(x_n)]} - 1\right)$$

b) Since it is linearly separable, you can set weighted misclassification error to 0. $\epsilon_1 = 0$, substitute

$$\beta_1 = \frac{1}{2} \ln\left(\frac{1}{0} - 1\right) = \boxed{\infty}$$

2) a) The optimal clustering is $\{1,2\}, \{5\}, \{7\}$. $\mu_1 = 1.5$, $\mu_2 = 5$, $\mu_3 = 7$

$$J(\{r_{nk}\}, \{\mu_k\}) = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|x_n - \mu_k\|_2^2 = \sum_{n=1}^{4} \sum_{k=1}^{3} r_{nk} \|x_n - \mu_k\|_2^2$$

$$= (1-1.5)^2 + (2-1.5)^2 + (5-5)^2 + (7-7)^2 = \boxed{\frac{1}{2}}$$

It makes sense to pair 1+2 together since they're the closest datpoints.

b) Let $\mu_1 = 1$, $\mu_2 = 2$, $\mu_3 = 6$. $\{1\}, \{2\}, \{5,7\}$.

Cost is: $= (1-1)^2 + (2-2)^2 + (5-6)^2 + (7-6)^2 = 2$.

$2 < \frac{1}{2}$, this is suboptimal compared to a.

It cannot be improved by changing the points to different clusters because no point is closer to another centroid than its current one.

1, 2, 5, 7 all will not change clusters.

changing centroids will not improve the objective either. Because, If we recomputed the centroids to minimize J more, they'd remain 1, 2 and 6 ((5+7)/2 = 6). Neither recomputing the centroids nor reassigning points works, so the clusters will not be improved. (and the algorithm converges here)

3) a) since μ_j is not present in the first term, and we're differentiating with respect to μ_j, we can focus only on the second term. we can also focus on only the jth term of the second term sum. can substitute the Pdf in:

$$\sum_n \gamma_{nj} \log\left(\frac{1}{(2\pi)^{d/2}|\Sigma_j|^{1/2}} e^{-1/2(x_n-\mu_j)^T \Sigma_k^{-1}(x_n-\mu_j)}\right)$$

$$\sum_n \gamma_{nj}\left(\log \underbrace{\frac{1}{(2\pi)^{d/2}|\Sigma_j|^{1/2}}}_{\text{doesn't depend on } \mu_j} - \frac{1}{2}(x_n-\mu_j)^T \Sigma_k^{-1}(x_n-\mu_j)\right)$$

$$\sum_n \gamma_{nj}\left(-\frac{1}{2}\underbrace{(x_n-\mu_j)^T}_{x^T}\underbrace{\Sigma_j^{-1}}_{A}\underbrace{(x_n-\mu_j)}_{x}\right) \qquad (A+A^T)x$$

apply gradient identity: $\nabla(x^T A x) =$

~~$= \frac{1}{2}(2\Sigma_k^{-1}(x_n-\mu_j))=$~~

$$= -\frac{1}{2}\sum_n \gamma_{nj}\left(\underbrace{\Sigma_j^{-1} + (\Sigma_j^{-1})^T}_{\text{equal since symmetric}}\right)(x_n-\mu_j)(-1)$$

$$= \boxed{\sum_n \gamma_{nj}\Sigma_j^{-1}(x_n-\mu_j)}$$

b) $\sum_n \gamma_{nj} \Sigma_j^{-1}(x_n - \mu_j) = 0$

$\Sigma_j^{-1}\left[\sum_n \gamma_{nj} x_n - \mu_j \sum_n \gamma_{nj}\right] = 0$

$\sum_n \gamma_{nj} x_n = \mu_j \sum_n \gamma_{nj}$

$$\boxed{\frac{1}{\sum_n \gamma_{nj}} \sum_n \gamma_{nj} x_n = \mu_j}$$

c) $\{5, 15, 25, 30, 40\}$ $\quad w_k = \dfrac{\sum_n \gamma_{nk}}{\underbrace{\sum_k \sum_n \gamma_{nk}}_{=N}} = \dfrac{\sum_n \gamma_{nk}}{N}$

$w_1 = \dfrac{0.2+0.2+0.8+0.9+0.9}{5} = \boxed{0.6}$

$w_2 = \dfrac{0.8+0.8+0.2+0.1+0.1}{5} = \boxed{0.4}$ $\quad \mu_k = \dfrac{\sum_n r_{nk} x_n}{\sum_n r_{nk}}$

$\mu_1 = \dfrac{(0.2)(5)+(0.2)(15)+(0.8)(25)+(0.9)(30)+(0.9)(40)}{0.2+0.2+0.8+0.9+0.9}$

$= \boxed{29}$

$\mu_2 = \dfrac{(0.8)(5)+(0.8)(15)+(0.2)(25)+(0.1)(30)+(0.1)(40)}{0.8+0.8+0.2+0.1+0.1} = \boxed{14}$

Solution: On Paper

4 Implementation: Clustering and PCA [32 pts]

4.1 PCA and Image Reconstruction [4 pts]

Before attempting automated facial recognition, you will investigate a general problem with images. That is, images are typically represented as thousands (in this project) to millions (more generally) of pixel values, and a high-dimensional vector of pixels must be reduced to a reasonably low-dimensional vector of features.

(a) **(1 pts)** As always, the first thing to do with any new dataset is to look at it. Use `get_lfw_data(...)` to get the LFW dataset with labels, and plot a couple of the input images using `show_image(...)`. Then compute the mean of all the images, and plot it. (Remember to include all requested images in your writeup.) Comment briefly on this "average" face.

```
X,y = get_lfw_data()
show_image(X[1])
show_image(X[15])
show_image(np.mean(X,axis=0))
```

Figure 1: Part 1A

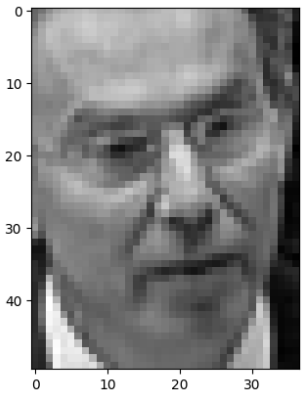


Figure 2: Face 1

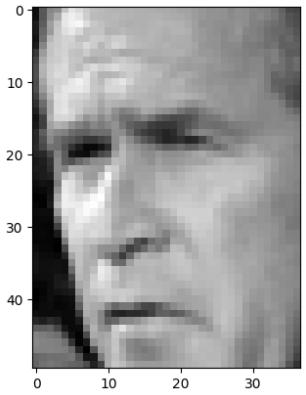


Figure 3: Face 2

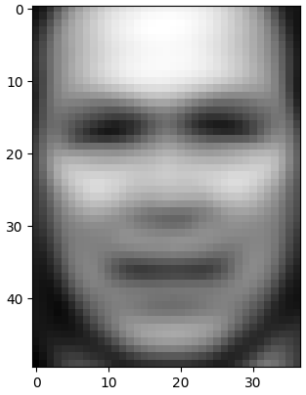


Figure 4: Average Face

Solution: This average face looks very generic and it is hard to recognize compared to the few individual images we plotted earlier. We see a blurring of key features that would help identify a face, such as bone structure, eye shape, mouth shape, etc. Instead, there is a vague figure of these parts of the face in their places.

(b) **(1 pts)** Perform PCA on the data using `util.PCA(...)`. This function returns a matrix `U` whose columns are the principal components, and a vector `mu` which is the mean of the data. If you want to look at a principal component (referred to in this setting as an eigenface), run `show_image(vec_to_image(v))`, where `v` is a column of the principal component matrix. (This function will scale vector `v` appropriately for image display.) Show the top twelve eigenfaces:

Comment briefly on your observations. Why do you think these are selected as the top eigenfaces?

```
U,mu = util.PCA(X)
plot_gallery([vec_to_image(U[:,i]) for i in range(12)])
```

Figure 5: Part 1B



Figure 6: Top 12 Eigenfaces

Solution: I think these are the top eigenfaces since they show large variations across the general features in the dataset. For example, we see the lighting and shadows go from very bright to very dark. It makes sense that the top eigenfaces represent the overall variance across pictures. Capturing these features and this variance is important when recognizing faces and making distinctions.

(c) **(2 pts)** Explore the effect of using more or fewer dimensions to represent images. Do this by:

- Finding the principal components of the data
- Selecting a number l of components to use
- Reconstructing the images using only the first l principal components
- Visually comparing the images to the originals

To perform PCA, use `apply_PCA_from_Eig(...)` to project the original data into the lower-dimensional space, and then use `reconstruct_from_PCA(...)` to reconstruct high-dimensional images out of lower dimensional ones. Then, using `plotGallery(...)`, submit a gallery of the first 12 images in the dataset, reconstructed with l components, for $l = 1, 10, 50, 100, 500, 1288$. Comment briefly on the effectiveness of differing values of l with respect to facial recognition.

```
for l in [1,10,50,100,500,1288]:
  Z,Ul = apply_PCA_from_Eig(X,U,l,mu)
  X_rec = reconstruct_from_PCA(Z,Ul,mu)
  plot_gallery(X_rec[:12])
```

Figure 7: Part 1C



Figure 8: l = 1



Figure 9: l = 10



Figure 10: l = 50



Figure 11: l = 100



Figure 12: l = 500



Figure 13: l = 1288

Solution: As l, or the number of principal components being used, increases, the images become more clear. When l = 500, I can clearly recognize who some of the images are, versus when l = 1, and it is hard to distinguish the images from one another. This is due to the inclusion of more components. Even with only 500 components, which is not even half of the dimensions in the last one (1288), we can recognize who the images are, signifying the importance of principal components.

We will revisit PCA in the last section of this project.

4.2 K-Means and K-Medoids [16 pts]

Next, we will explore clustering algorithms in detail by applying them to a toy dataset. In particular, we will investigate k-means and k-medoids (a slight variation on k-means).

(a) **(1 pts)** In k-means, we attempt to find k cluster centers $\boldsymbol{\mu}_j \in \mathbb{R}^d$, $j \in \{1, \ldots, k\}$ and n cluster assignments $c^{(i)} \in \{1, \ldots, k\}$, $i \in \{1, \ldots, n\}$, such that the total distance between each data point and the nearest cluster center is minimized. In other words, we attempt to find $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ and $c^{(1)}, \ldots, c^{(n)}$ that minimizes

$$J(\boldsymbol{c}, \boldsymbol{\mu}) = \sum_{i=1}^{n} ||\boldsymbol{x}^{(i)} - \boldsymbol{\mu}_{c^{(i)}}||^2.$$

To do so, we iterate between assigning $\boldsymbol{x}^{(i)}$ to the nearest cluster center $c^{(i)}$ and updating each cluster center $\boldsymbol{\mu}_j$ to the average of all points assigned to the jth cluster.

Instead of holding the number of clusters k fixed, one can think of minimizing the objective function over $\boldsymbol{\mu}$, $\boldsymbol{c}$, *and* k. Show that this is a bad idea. Specifically, what is the minimum possible value of $J(\boldsymbol{c}, \boldsymbol{\mu}, k)$? What values of $\boldsymbol{c}$, $\boldsymbol{\mu}$, and k result in this value? **Solution:** We would minimize the total distance between each datapoint and the nearest cluster center by having each datapoint be its own cluster. This is not ideal since it would likely lead to overfitting rather than learning generalizable patterns in the data. The minimum possible value of J is 0. In this case, c(i) = i for all i in 1,...,n, $\mu = x^j$ all j in 1,...,n, and k = n, the number of datapoints since we have one cluster for each point.

(b) **(3 pts)** To implement our clustering algorithms, we will use Python classes to help us define three abstract data types: `Point`, `Cluster`, and `ClusterSet` (available in `cluster.py`). Read through the documentation for these classes. (You will be using these classes later, so make sure you know what functionality each class provides!) Some of the class methods are already implemented, and other methods are described in comments. Implement all of the methods marked `TODO` in the `Cluster` and `ClusterSet` classes. **Solution:**

```
labels = []
for p in self.points:
  labels.append(p.label)
centroid_label = np.argmax(np.bincount(labels))
centroid_attrs = np.mean([p.attrs for p in self.points],axis=0)
centroid = Point("centroid", centroid_label, centroid_attrs)
```

Figure 14: Cluster class, centroid

```
min_distance = float('inf')

for p in self.points:
  distance = sum(p.distance(other) for other in self.points)
  if distance < min_distance:
    min_distance = distance
    medoid = p

return medoid
```

Figure 15: Cluster class, medoid

```
centroids = []
for cluster in self.members:
  centroids.append(cluster.centroid())
return centroids
```

Figure 16: ClusterSet, centroids

```
medoids = []
for cluster in self.members:
  medoids.append(cluster.medoid())
return medoids
```

Figure 17: ClusterSet, medoids

(c) **(6 pts)** Next, implement `random_init(...)` and `kMeans(...)` based on the specifications provided in the code. **Solution:**

```
initial_points = np.random.choice(points, size=k, replace=False)
return initial_points
```

Figure 18: 4.2C random init

```
if init == "random":
  centers = random_init(points,k)
else:
  centers = cheat_init(points)

converged = False
i=1

k_clusters = ClusterSet()
while not converged:
  new_assignments = {}
  for c in centers:
    new_assignments[c] = []

  for p in points:
    closest_center = min(centers, key=lambda c: p.distance(c))
    new_assignments[closest_center].append(p)

  old_k_clusters = k_clusters
  k_clusters = ClusterSet()

  for assignment in new_assignments.values():
    k_clusters.add(Cluster(assignment))
  centers = k_clusters.centroids()
  print(f"Iteration {i}")
  converged = old_k_clusters.equivalent(k_clusters)

  if plot:
    plot_clusters(k_clusters,f"Iteration {i}", ClusterSet.centroids)
  i+=1
return k_clusters
```

Figure 19: kMeans

(d) **(1 pts)** Now test the performance of k-means on a toy dataset.

Use `generate_points_2d(...)` to generate three clusters each containing 20 points. (You can modify `generate_points_2d(...)` to test different inputs while debugging your code,

but be sure to return to the initial implementation before creating any plots for submission.) You can plot the clusters for each iteration using the `plot_clusters(...)` function.

In your writeup, include plots for the k-means cluster assignments and corresponding cluster "centers" *for each iteration* when using random initialization. **Solution:**

```
points = generate_points_2d(20)
kMeans(points,3,init='random',plot=True)
```

Figure 20: 4.2D Code

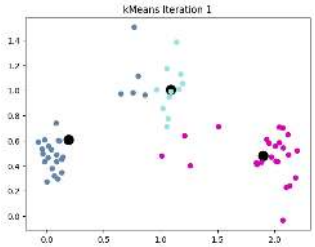


Figure 21: Iteration 1

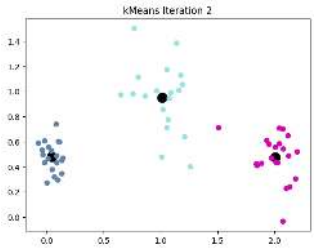


Figure 22: Iteration 2

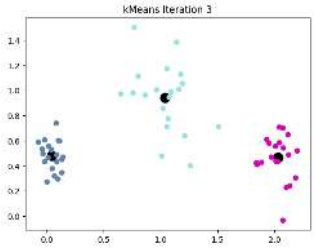


Figure 23: Iteration 3

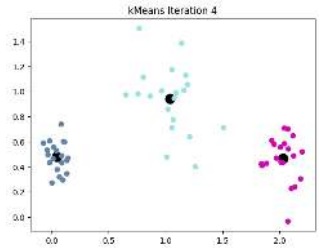


Figure 24: Iteration 4

(e) **(3 pts)** Implement `kMedoids(...)` based on the provided specification.

Hint: Since k-means and k-medoids are so similar, you may find it useful to refactor your code to use a helper function `kAverages(points, k, average, init='random', plot=True)`, where `average` is a method that determines how to calculate the average of points in a cluster (so it can take on values `ClusterSet.centroids` or `ClusterSet.medoids`).[1]

As before, include plots for k-medoids clustering *for each iteration* when using random initialization. **Solution:**

```
return kAverages(points,k,ClusterSet.medoids,init,plot)
```

Figure 25: kMedoids Function

```
if init == "random":
  centers = random_init(points,k)
else:
  centers = cheat_init(points)

converged = False
i=1

k_clusters = ClusterSet()
while not converged:
  new_assignments = {}
  for c in centers:
    new_assignments[c] = []

  for p in points:
    closest_center = min(centers, key=lambda c: p.distance(c))
    new_assignments[closest_center].append(p)

  old_k_clusters = k_clusters
  k_clusters = ClusterSet()

  for assignment in new_assignments.values():
    k_clusters.add(Cluster(assignment))
  centers = average(k_clusters)
  print(f"Iteration {i}")
  converged = old_k_clusters.equivalent(k_clusters)

  if plot:
    plot_clusters(k_clusters,f"Iteration {i}", average)
  i+=1
return k_clusters
```

Figure 26: kAverages

[1]In Python, if you have a function stored to the variable `func`, you can apply it to parameters `arg` by callling `func(arg)`. This works even if `func` is a class method and `arg` is an object that is an instance of the class.

```
kMedoids(points,3,init='random',plot=True)
```

Figure 27: kMedoids, random

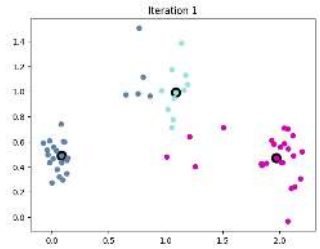


Figure 28: Iteration 1



Figure 29: Iteration 2

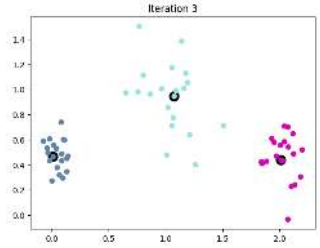


Figure 30: Iteration 3

(f) **(2 pts)** Finally, we will explore the effect of initialization. Implement `cheat_init(...)`. Now compare clustering by initializing using `cheat_init(...)`. Include plots for k-means and k-medoids *for each iteration*. **Solution:**

```
clusters = {}
for p in points:
  if p.label not in clusters:
    clusters[p.label] = []
  clusters[p.label].append(p)

initial_points = []
for value in clusters.values():
  medoid = Cluster(value).medoid()
  initial_points.append(medoid)

return initial_points
```

Figure 31: cheat init

```
kMeans(points,3,init='cheat',plot=True)
kMedoids(points,3,init='cheat',plot=True)
```

Figure 32: Cheat Code

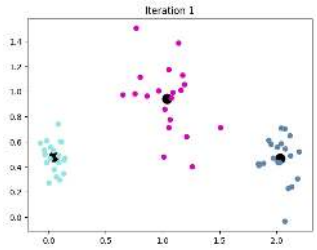


Figure 33: Cheat, kMeans, Iteration 1

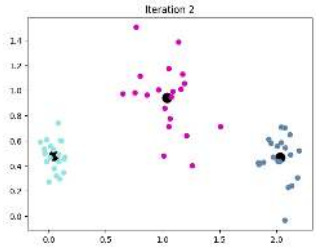


Figure 34: Cheat, kMeans, Iteration 2

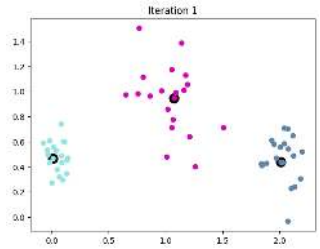


Figure 35: Cheat, kMedoids, Iteration 1

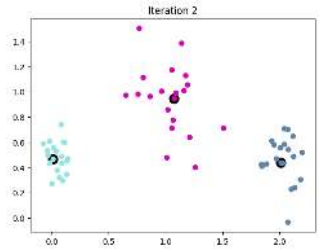


Figure 36: Cheat, kMedoids, Iteration 2

4.3 Clustering Faces [12 pts]

Finally (!), we will apply clustering algorithms to the image data. To keep things simple, we will only consider data from four individuals. Make a new image dataset by selecting 40 images each from classes 4, 6, 13, and 16, then translate these images to (labeled) points: [2]

(a) **(2 pts)** Apply k-means and k-medoids to this new dataset with $k = 4$ and initializing the centroids randomly. Evaluate the performance of each clustering algorithm by computing the average cluster purity with `ClusterSet.score(...)`. As the performance of the algorithms can vary widely depending upon the initialization, run both clustering methods 10 times and report the average, minimum, and maximum performance.

```
np.random.seed(1234) ## don't change the seed !!
X1, y1 = util.limit_pics(X, y, [4, 6, 13, 16], 40)
points = build_face_image_points(X1,y1)

mean_scores = []
med_scores = []
for i in range (1,11):
  mean_clusters = kMeans(points,4,init='random',plot=False)
  med_clusters = kMedoids(points,4,init='random',plot=False)
  mean_scores.append(mean_clusters.score())
  med_scores.append(med_clusters.score())
  i+=1

print(f"Mean Scores: {mean_scores}")
print(f"Medoid Scores: {med_scores}")
print(f"Mean Average: {np.mean(mean_scores)}, Medoid Average: {np.mean(med_scores)}")
print(f"Mean Min: {np.min(mean_scores)}, Medoid Min: {np.min(med_scores)}")
print(f"Mean Max: {np.max(mean_scores)}, Medoid Max: {np.max(med_scores)}")
```

Figure 37: 4.3A Code

[2] There is a bug in fetch_lfw version 0.18.1 where the results of the loaded images are not always in the same order. This is not a problem for the previous parts but can affect the subset selected in this part. Thus, you may see varying results. Results that show the correct qualitative behavior will get full credit.

Solution:

	average	min	max
k-means	0.583	0.519	0.688
k-medoids	0.628	0.543	0.756

How do the clustering methods compare in terms of clustering performance and runtime? **Solution:** Overall, k-medoids seems to perform better between the two since we have higher scores. The average is higher by 0.04

Now construct another dataset by selecting 40 images each from two individuals 4 and 13.

(b) **(5 pts)** Explore the effect of lower-dimensional representations on clustering performance. To do this, compute the principal components for the entire image dataset, then project the newly generated dataset into a lower dimension (varying the number of principal components), and compute the scores of each clustering algorithm.

So that we are only changing one thing at a time, use `init='cheat'` to generate the same initial set of clusters for k-means and k-medoids. For each value of l, the number of principal components, you will have to generate a new list of points using `build_face_image_points(...)`.

Let $l = 1, 3, 5, \ldots, 41$. The number of clusters $K = 2$. Then, on a single plot, plot the clustering score versus the number of components for each clustering algorithm (be sure to label the algorithms). Discuss the results in a few sentences. **Solution:** With under 4 principal components, KMedoids performs worse than KMeans does. After that until 41, KMedoids is consistently better than KMeans. KMedoids did oscillate up and down a bit more than KMeans, but from 15-41, Kmedoids remained much higher in score than KMeans. With more components, KMedoids got better but KMeans did not see as much improvement.

```
np.random.seed(1234) ## don't change the seed !!
X2, y2 = util.limit_pics(X, y, [4, 13], 40)
scores1 = []
scores2 = []
for l in range(1,42,2):
  Z, Ul = apply_PCA_from_Eig(X2,U,l,mu)
  points = build_face_image_points(Z,y2)
  clusters1 = kMeans(points,2,init='cheat',plot=False)
  clusters2 = kMedoids(points,2,init='cheat',plot=False)
  scores1.append(clusters1.score())
  scores2.append(clusters2.score())

l = [i for i in range(1,42,2)]
plt.plot(l,scores1,label='kMeans')
plt.plot(l,scores2,label='kMedoids')
plt.xlabel('Number of Principal Components')
plt.ylabel('Scores')
plt.legend()
plt.show()
```

Figure 38: 4.3B Code

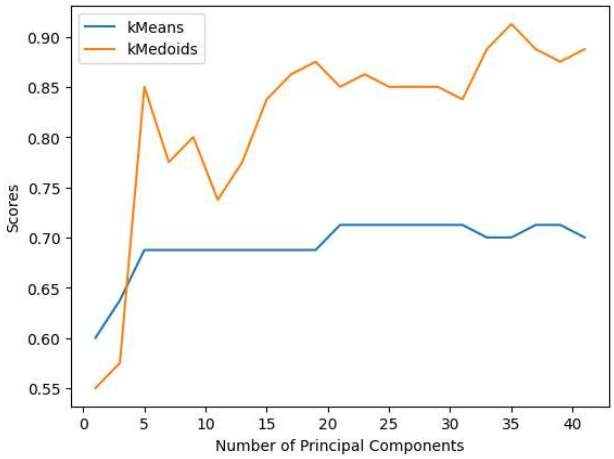


Figure 39: 4.3B

Some pairs of people are more similar to one another and some more different.

(c) **(5 pts)** Experiment with the data to find a pair that clustering can discriminate very well and another pair that it finds very difficult (assume you have 40 images for each individual). Describe your methodology (you may choose any of the clustering algorithms you implemented). Report the two pairs in your writeup (display the pairs of images using `plot_representative_images`), and comment briefly on the results. **Solution:** I used KMeans and played around with testing the score of the results with different individuals. I got a score of 0.9875 for individuals 9 and 16, meaning they were most discriminative and their clusters were very different. I got a score of 0.5 for 0 and 1, meaning they were one of the least discriminative pairs and clusters were harder to separate. When looking at the images, this makes sense. Also, the less discriminative pair took way more iterations, which makes sense with my claim that they are harder to distinguish. These were the lowest and highest purities I found across pairs.

```
np.random.seed(1234) ## don't change the seed !!

X3, y3 = util.limit_pics(X,y,[9,16],40)
points = build_face_image_points(X3,y3)
clusters = kMeans(points,2,init='random',plot=False)
print(clusters.score())
plot_representative_images(X,y,labels=[9,16])

X4, y4 = util.limit_pics(X,y, [0,1], 40)
points = build_face_image_points(X4,y4)
clusters = kMeans(points,2,init='random',plot=False)
print(clusters.score())
plot_representative_images(X,y,labels=[0,1])
```

Figure 40: 4.3C Code



Figure 41: 9 vs 16, score = 0.9875



Figure 42: 0 vs 1, score = 0.5